Computer Vision – AIDAMS
ESSEC Business School & CentraleSupélec

Emoji Provider Classification with a Convolutional Neural Network

Victor Henz
Jawad Mallat
Adem Ben Njima
Edgar Gusching

Kaggle team: Jawad Mallat
Instructor: Titouan Rigaud

October 28, 2026

Abstract

We classify emoji images by provider (7 classes) and treat it as a style recognition problem. To start, we chose a 4-block convolutional neural network trained from scratch, a deliberately simple choice that we can explain layer by layer. Methods like regularisation (batch normalisation, dropout and label smoothing) raised accuracy from about 86% to 92%, while data augmentation lowered it, since test emojis are never rotated or recoloured. The biggest gain came not from the architecture, but from the input: keeping the transparency (alpha) channel and adding each image's original size and format brought accuracy to about 95% and cut Facebook and WhatsApp confusions from 38 to 6. Our best Kaggle score is 0.94877, and a second run of the same notebook gave almost the same result (0.94857), so it is not just a lucky run. Our main takeaway is that understanding the data mattered more than making the model bigger.

Contents

1 Introduction **2**

2 Model Chosen **2**

3 Preprocessing **2**

4 Architecture **3**

5 Metrics **4**

6 Performance **5**

7 AI and Code **7**

8 Conclusion **8**

1 Introduction

Send the same emoji from an iPhone and from an Android phone, and the person receiving it may not see the same picture. Each platform has its own version: the same expression, the same meaning, but a different design. A trained eye can often tell them apart, but how will a machine perform? That is the goal of this project.

Given an emoji image, the task is to predict which provider drew it among seven classes: Apple, Facebook, Google, Messenger, Mozilla, Samsung and WhatsApp. The real difficulty is that the content of the image is no help, since two versions of the same emoji represent the exact same thing. The model has to learn another way. How can a human spot the difference between these emojis? The answer lies in the style: different colour palettes, shading, outlines and proportions. Additionally, some providers are so close that they are easily confused. For example, Facebook and WhatsApp were by far our worst pair, with errors adding up to 38 at one point.

The dataset contains 9,879 training images and 9,879 test images. The images were originally 72, 160 or 240 pixels wide, and we resized them to 72×72. When digging deeper into the data, we found a key caveat that we had to keep in mind: the classes are imbalanced. The five largest classes have between about 1,650 and 1,925 images each, against only 580 for Messenger and 397 for Mozilla. Our evaluation metric is accuracy on the Kaggle test set. We also wanted to make sure the computation did not get too heavy. This trade-off between performance and computation was essential in our decision making throughout the whole project.

2 Model Chosen

We had a choice to make: either design our own network or reuse a large pre-trained one. We chose to build a convolutional neural network (CNN) from scratch. CNNs are the standard tool for images, they are the family we studied in class, and building one ourselves meant we could explain every layer.

Ours has one particularity: besides the usual colour channels, it receives the transparency channel, the original size and the format of each image, added just before the final classifier. To test our choice, we also fine-tuned a pre-trained ResNet-18. It did score higher, but only once our 72-pixel emojis were enlarged to 144 or 224 pixels, at a much higher compute cost. Its advantages came from features learned on millions of external photos, rather than from anything we designed. We kept our own CNN, which is described in Section 4.

3 Preprocessing

The notebook template loaded every image with `convert("RGB").resize((72,72))`, and stated that this preprocessing was incorrect on purpose: finding the right one was part of the challenge. With it, our CNN stalled at about 92.5% independently of what we changed in the model.

Going back to the raw PNG files, we found two pieces of information this loader throws away:

1. **The original size and format.** Each provider exports its emojis at its own size: Facebook in 240×240, WhatsApp in 160×160, and Apple, Google and Samsung in 72×72. Size alone separates Facebook from WhatsApp 96.5% of the time, but resizing everything to 72×72 erased it. The file format (colour, palette or grayscale, with or without transparency) also varies between providers, though less clearly.

2. **The transparency.** Every emoji sits on a transparent background. Even a fully transparent pixel still stores a colour that we never see, and this colour depends on the provider: mostly black for Mozilla, Google and Apple, rarely for Samsung. Converting to RGB removed the

transparency and revealed these hidden colours. This is why many emojis appeared on a black square, which we first mistook for a shared drawing style.

We decided for the loader to keep both. Before resizing, it stores each image's size and format as a 9-value one-hot vector. It then resizes the color channels and the transparency channel separately, since resizing them together turns every transparent pixel black, and stacks them into a $72 \times 72 \times 4$ image with pixel values scaled from 0–255 to 0–1. To measure accuracy on images the model never saw, we keep 20% of the training set for validation, with the same share of each provider in both sets.

Table 1: Effect of the input (same model, training and validation set).

Input	Val. accuracy	Facebook↔WhatsApp errors
RGB only (template loader)	92.5%	38
Background painted white + transparency	90.8%	14
RGB + transparency	93.9%	23
RGB + size/format	94.6%	9
RGB + transparency + size/format (kept)	**95.3%**	**4**

Each piece of information helps on its own, and together they bring accuracy from 92.5% to 95.3%. Most of the gain comes from Facebook and WhatsApp, whose confusions drop from 38 to 4, and consequently accuracy rises. Painting the background white does the opposite: it erases the hidden colours, and accuracy falls to 90.8%. Apple and Google remain our hardest pair (about 20 errors, against 30 before): both use 72×72 files with transparency, so only their drawings can tell them apart.

Finally, we tried data augmentation through flips, color changes, and small shifts, all of which lowered accuracy by up to 2.5 points. This makes sense because a provider is recognised by its style, including the position and colors of the emoji, which is exactly what these transformations change. Since test emojis are never flipped or recolored, we did not use any.

4 Architecture

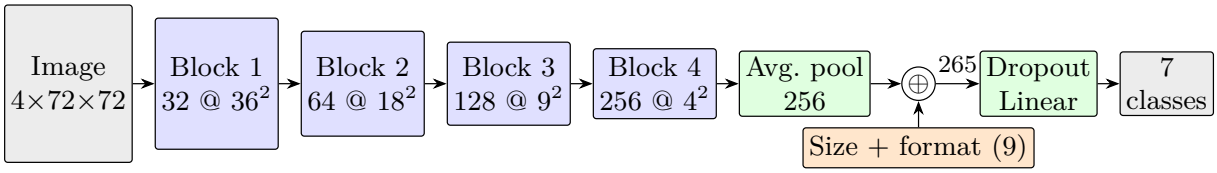


Figure 1: Our CNN (1.18M parameters). Each block: [Conv 3×3 → BatchNorm → ReLU] ×2 → MaxPool 2×2 → Dropout2d.

Our network is built from four similar blocks (Figure 1). Each block applies two small 3×3 filters, then halves the image. As the image shrinks from 72 to 4 pixels, the number of channels grows from 32 to 256: the first blocks detect simple details such as outlines and shading, the last ones whole parts of the emoji. Each block also uses batch normalisation, which keeps training stable, and dropout, which randomly switches off parts of the network so that it cannot rely on a few features.

At the end, each of the 256 channels is averaged into a single number. This is where the image meets the file information: we add the 9 size and format values to these 256 features, and a final layer turns the 265 values into a score for each of the 7 providers. We add the file information this late because it is not part of the picture, so there is nothing for the filters to look at. In total, the model has 1.18 million parameters.

Table 2: Architectures we tried. "Full" = RGB + transparency + size/format. Time per epoch: 9 s for our CNN, 8/27/62 s for pre-trained ResNet-18 at 72/144/224 px.

Model	Input	Params	Val. accuracy
First CNN (2 conv + dense)	RGB	>1.3M	86%
4-block CNN	RGB	1.18M	92.6%
+ 5th block	RGB	4.7M	91.9%
+ attention layer	RGB	–	90.0%
4-block CNN (final)	**full, 72 px**	**1.18M**	**94.8–95.6%**
ResNet-18 from scratch	full, 72 px	11.2M	88.9%
ResNet-18 pre-trained	full, 72/144/224 px	11.2M	93.2/96.1/97.3%

This table is the fruit of weeks of discussions and tests to improve accuracy. Our first model memorised the training images and got 99% accuracy on them but only 86% on validation. Most of its parameters sat in one large final layer, which the averaging step removed. When the 4-block model then stalled at about 92.6% (the plateau we reached before realizing that preprocessing would unlock the Whatsapp <–> Facebook confusion), we thought the model didn't have enough layers and thus tried to add a fifth block and an attention layer. We then decided to remove both because the accuracy had dropped. ResNet-18 only beat us when pre-trained and given enlarged images, at 3 to 7 times our training time.
We decided to keep this model for the final submission because of its simplicity (9 layers, against 18 for ResNet-18), the fact that it was trained exclusively on our data, and its ability to generalize: although training accuracy is about 3 points higher than validation accuracy (98.6% vs 95.5%), validation accuracy never decreased during training, and our Kaggle score on unseen test images (94.9%) matches it.

5 Metrics

The competition is graded on accuracy, so accuracy is our reference and the number we optimised. On its own, however, it summarises this dataset poorly.

Because the class sizes are so uneven (Section 1), a provider can fail almost completely without moving it. Mozilla is 4% of the images, so losing every Mozilla emoji would cost four points of accuracy and still leave a respectable leaderboard score. Accuracy alone cannot tell us whether we built a model that recognises seven providers, or one that recognises five and guesses on the other two.

We therefore compute three additional measures from the validation confusion matrix, writing TP_c, FP_c and FN_c for the true positives, false positives and false negatives of a provider c.

1. **Precision and recall, per provider.** Recall is the share of a provider's emojis that we found; precision is the share of our predictions for that provider that were correct. In the confusion matrix, recall divides the diagonal entry by its row total and precision divides it by its column total.
$$R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}, \qquad P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}.$$

2. **F1, per provider.** A single score per provider, combining the two above through a harmonic mean rather than an ordinary one, so that it stays close to whichever of them is lower.
$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}.$$

3. **Macro F1 and balanced accuracy.** Both average over the $K = 7$ providers instead of

over the 9,879 images, so Mozilla counts as much as Apple.

$$\text{macro } F_1 = \frac{1}{K}\sum_c F_{1,c}, \qquad \text{balanced accuracy} = \frac{1}{K}\sum_c R_c.$$

The difference between these measures is what they average over, and it decides which failures stay visible. Accuracy sums over images, so a provider contributes in proportion to its size and a small class can collapse almost unnoticed. Macro F1 and balanced accuracy sum over providers, giving each one weight 1/7 whatever its size, so the gap between them and accuracy measures directly what the imbalance costs us.

Precision and recall matter separately because they fail in opposite directions. A rare provider the model hesitates to predict loses recall while keeping precision, and whichever provider absorbs it loses precision while keeping recall. Reported as a single number, each of those is easy to obtain and meaningless on its own: predicting one provider everywhere gives it perfect recall, and predicting it once gives it perfect precision. Together, and summarised by $F_{1,c}$, they describe the exchange.

Table 3: The measures we report and the question each one answers.

Measure	Question it answers	Averaged over
Accuracy	How many images did we get right?	images
Recall R_c	Of the real c emojis, how many did we find?	—
Precision P_c	When we predict c, are we right?	—
Macro F1	How well do we handle a provider?	providers
Balanced accuracy	Mean recall, every provider counted equally	providers

These two families pull in different directions, and we chose between them deliberately. Weighting the loss by inverse class frequency raised Mozilla recall substantially but cost roughly a point of overall accuracy, by making the model less decisive on the majority providers. Since the grade is plain accuracy, we left the weighting out, but we report both families below so the cost of that choice stays visible.

6 Performance

Our best Kaggle score is 0.94877. Re-running the same notebook gave 0.94857, a difference of about two images out of 9,879, so the result is stable rather than a lucky run.

On the held-out validation set of 1,976 images, the same model reaches 95.09% accuracy. That the two agree is worth noting on its own: the validation split guided our choice of preprocessing, architecture and hyperparameters, so it could easily have become optimistic, and it did not.

Table 4: Per-provider performance on the validation set (1,976 images), in %.

Provider	Support	Precision	Recall	F1	Errors
apple	385	93.6	95.1	94.3	19
facebook	333	94.8	93.7	94.3	21
google	376	95.3	97.3	96.3	10
messenger	116	82.8	95.7	88.8	5
mozilla	79	96.7	73.4	83.5	21
samsung	358	98.6	98.3	98.5	6
whatsapp	329	97.8	95.4	96.6	15
Accuracy 95.09		Balanced accuracy 92.71		Macro F1 93.18	

Samsung is close to solved at 98.5 F1, and Google follows. The outlier is Mozilla, whose recall is 73.4%: we miss 21 of its 79 images, and 17 of those are predicted as Messenger. That single cell is the largest error in the matrix, larger than the Apple and Google confusion we had assumed was our main remaining weakness.

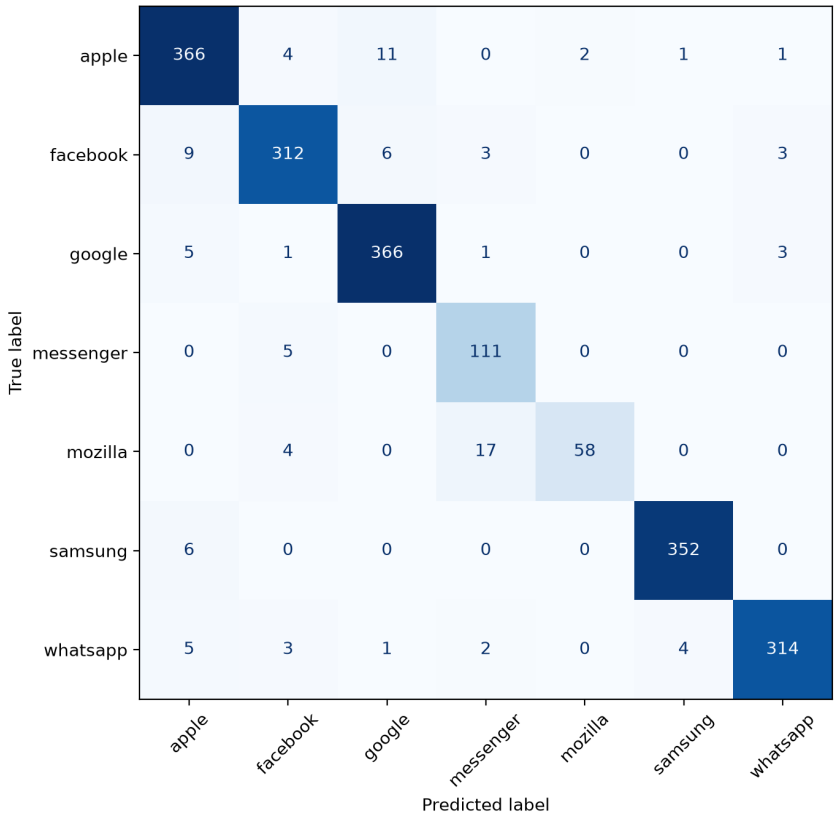


Figure 2: Validation confusion matrix. Rows are true providers, columns predictions.

The direction of the failure is as informative as its size. Mozilla combines high precision (96.7%) with low recall: when the model commits to Mozilla it is almost always right, but it rarely commits. Messenger is the mirror image, with the worst precision of the seven (82.8%) alongside high recall.

Both numbers describe one event. Our rarest provider is being absorbed by a larger neighbour, and 79 images are not enough for the model to predict it confidently. Accuracy records this as roughly one point; the difference between 95.09 and 92.71 records it as what it is.

Read across the whole matrix, the errors that remain share one structure. Mozilla and Messenger are both exported at 240×240, and Apple and Google are both 72×72 with transparency. In each case the file properties that resolved Facebook and WhatsApp (down to six errors, from 38) separate nothing, and only the drawing itself distinguishes the two providers. Our metadata features did not make the task easy: they removed the confusions they could resolve and left exactly those they could not.

Training accuracy at the restored best epoch is 98.1% against 95.1% on validation, a gap of three points. The gap alone does not settle the question of overfitting. What does is that validation loss fell from epoch 13 to the end of training without ever turning upward, and that validation accuracy held between 94.9% and 95.1% across the final five epochs. A model beginning to memorise shows validation loss rising while training loss keeps falling, and we never observe that. The Kaggle score, computed on images from neither set, confirms it from outside. Absolute loss values are floored by label smoothing and are not comparable to an unsmoothed run.

Whether we stopped too early, or too small, is the more interesting question. Capacity is not the limit: a fifth block and an attention layer both lowered validation accuracy rather than raising it (Section 4), which is the opposite of what a model short of capacity does. Training length is not the limit either, since the one-cycle schedule anneals the learning rate to zero and the last five epochs are flat. The run converged.

The pre-trained ResNet-18 does reach 97.3%, so headroom exists. It needs ImageNet weights and images upscaled to 224 pixels to find it, at seven times our training cost, which means that headroom comes from external data and resolution rather than from anything our compute budget denied us. At 72 pixels, trained on this dataset alone, we appear to be near the ceiling.

The one place we knowingly leave performance behind is Mozilla. Weighting the loss by inverse class frequency raises its recall sharply and costs about a point of overall accuracy. Under the competition metric that is a poor exchange and we did not make it; under macro F1 it is a good one. Reporting both is what makes the choice visible: this model is tuned for the leaderboard, and one that had to treat all seven providers equally should be trained differently.

7 AI and Code

Collaboration. To coordinate our work, we created a GitHub repository with five branches: the main branch and one branch per group member. We met every week to discuss individual progress and merged the most convincing approach into the main branch. The repository is available at `https://github.com/Jawadmallat/computer_vision_project`.

Declaration of AI use

Jawad

My role. I wrote the preprocessing section (Section 3) and the loader that keeps the transparency channel and the original size and format of each image.

AI in the code. I used Claude Opus 5.5 throughout the project to write code and to suggest new methods. The most substantial breakthrough came after seeing that Facebook and WhatsApp were our worst pair (38 errors), and understanding that comparing size and format solved the issue . I tested each idea on the validation set (for example, size and format reduced the Facebook-WhatsApp errors from 38 to 4). To understand the code, I went through the notebook and asked the model to explain it cell by cell.

Victor

My role. I wrote the abstract, the introduction and "Model chosen" (Sections 1 and 2). On the code side, I worked on regularisation (batch normalisation, dropout, label smoothing), tested data augmentation, and built a three-model ensemble.

AI in the report. I used Claude (Anthropic) to plan Sections 1 and 2. I wrote every draft myself, then asked Claude for feedback on flow and factual errors, reviewed each suggestion and rewrote the final text. Claude also converted the text to LaTeX and set up the Overleaf template. I dictated my drafts with Wispr Flow, an AI speech-to-text tool.

AI in the code. I read the Keras documentation from class, pointed Opus 5.5 to it, asked which techniques could improve our model, and asked it to implement them. I checked each change on the validation set: regularisation raised accuracy from 0.855 to 0.923 and was kept, while random erasing lowered it and was dropped.

Edgar

I used Claude Opus 5.5 to help me understand parts of the codebase written by my teammates, especially sections too complex to grasp on my own, and Claude Sonnet 5.5 to assist with writing the report in Overleaf: improving the fluency and structure of my paragraphs and checking that figures and citations followed the correct LaTeX formatting. The technical content, interpretations

and conclusions presented in my sections remain my own, and I verified that I understood every explanation before relying on it.

Adem

I used GPT Astra to help me understand the more complex parts of the code written by my teammates and to suggest and implement different techniques aimed at improving the prediction score. I tested these changes and compared their performance on the validation set. I also used it to review my draft of the final part of the report, correcting the language, improving the clarity and fluency of my paragraphs, and converting the text into LaTeX for Overleaf. ‘

8 Conclusion

We built a four-block CNN from scratch to recognise the provider of an emoji by its style. Global average pooling, batch normalisation, dropout and label smoothing helped raise validation accuracy from about 86% to 92.6%. Adding a fifth block or an attention layer lowered accuracy, so increasing model complexity did not improve our results.

The largest gain came from the input. The template loader discarded the transparency channel and the original size and format of each file, which help distinguish some providers. Keeping them raised validation accuracy from 92.5% to 95.3% in our input comparison, and cut Facebook and WhatsApp confusions from 38 to 6 in the final model. Data augmentation lowered accuracy, since it alters the colours and positions that help define a provider's style.

The final model reached 94.877% on Kaggle, with 94.857% on a second run, consistent with its 95.09% validation accuracy. Per-provider metrics showed what accuracy hides: Mozilla recall was only 73.4%, and 17 of its 21 missed images were predicted as Messenger. This weakness contributed to the gap between accuracy (95.09%) and balanced accuracy (92.71%). Class weighting improved Mozilla recall but cost about one point of overall accuracy, so we left it out for the competition.

A pre-trained ResNet-18 reached 97.3% validation accuracy, but required ImageNet weights, 224-pixel inputs and seven times our training time. We therefore kept our simpler CNN as a balance between accuracy, simplicity and computational cost. Overall, understanding the data mattered more than making the model bigger.